Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

School of Physics

An Investigation of Graph Neural Networks for Molecular Property Prediction

Eoghan Stafford

Supervisor: Prof. Alessandro Lunghi

2025–2026

Abstract

This report investigates the viability of graph neural networks (GNN) for molecular property prediction. Two GNN architectures of increasing geometric complexity were developed, using atoms as node features and interatomic interactions as edge features. The two-body GNN encodes pairwise interatomic distances as edge features, while the three-body GNN additionally incorporates angular information, providing the model with a more complete representation of the local molecular geometry. Both architectures were trained to predict the potential energy of a set of benchmark molecules from the revised MD17 dataset and compared against a classical feedforward neural network trained on hand-crafted bond length and angle descriptors.

The three-body GNN consistently outperforms the two-body GNN across all molecules, confirming the value of angular encoding for predictive capabilities. The feedforward neural network remains competitive for molecules with uniform bonding environments but struggles with increasing molecular complexity, where fixed geometric descriptors fail to fully capture the electronic behavior of differing bond types. The three-body GNN is trained on an increased training set size which is shown to reduce overfitting and improve generalisation across all molecules. Per atom energy contributions were extracted from the three-body GNN for benzene and are found to be consistent with the known electronic structure of the molecule, validating that the model is learning physically meaningful atomic representations.

Acknowledgments

I would like to express my gratitude to my supervisor, Professor Alessandro Lunghi, for his guidance and encouragement throughout this project. His willingness to support a coding-heavy research project despite my limited programming background at the outset is something I am truly grateful for.

I would like to extend a particular thanks to Dr. Zahra Khatibi, whose patience, guidance, and encouragement were a constant throughout this work. Her willingness to meet regularly and offer support at every stage of the project made a considerable difference. Her confidence in my ability to undertake new challenges was something I came to rely on greatly.

Finally, I would like to acknowledge Lion Frangoulis, whose master's thesis served as the foundational reference for my understanding of graph neural networks and their implementation for molecular property prediction. The approach taken in this work owes much to the framework he established, and his willingness to offer advice and suggestions on the overall project was greatly appreciated.

Declaration

I declare this report is entirely my own work, except where stated through references or in the acknowledgments, and has not been submitted for any other degree or qualification. All sources used in the report have been cited and referenced in accordance with the submission guidelines for PYU44TP1. Generative AI tools were used to assist in the display of some figures within this report.

I have read and understood the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at `http://www.tcd.ie/calendar`.

I have also read and understood the guide, and completed the 'Ready Steady Write' Tutorial on avoiding plagiarism, located at `https://libguides.tcd.ie/academic-integrity/ready-steady-write`.

March 31, 2026

Contents

1 Motivation

Predicting molecular properties such as total ground state energy is a fundamental problem of quantum chemistry. In principle this requires solving the many body Schrodinger equation for a Hamiltonian that encodes the kinetic energy of all electrons and nuclei alongside every pairwise interaction between them.

$$\hat{H} = -\sum_{i=1}^{N_e} \frac{1}{2}\nabla_i^2 - \sum_{A=1}^{N_A} \frac{1}{2M_A}\nabla_A^2 - \sum_{i,A} \frac{Z_A}{|\vec{r}_i - \vec{R}_A|} + \sum_{i<j} \frac{1}{|\vec{r}_i - \vec{r}_j|} + \sum_{A<B} \frac{Z_A Z_B}{|\vec{R}_A - \vec{R}_B|} \quad (1)$$

Exact analytical solutions only exist for one-electron systems such as the Hydrogen atom. The presence of the electron-electron term in multi-electron systems destroys the separability of the problem, making analytical solutions impossible leading to a sequence of numerical approximations.

In practice, quantum chemistry relies on a hierarchy of numerical methods to approximate molecular energies and related properties. Hartree-Fock treats the problem as a mean field approximation, where each electron experiences an average potential from the other electrons in the system which is solved iteratively through the Roothaan-Hall equations [1]. Density functional theory (DFT) reformulates [2] the problem in terms of electron density rather than the many body wavefunction. Higher accuracy can be obtained through correlated wavefunction methods such as coupled-cluster theory (CCSD(T)) [3] which accounts for electron correlation beyond a mean field approximation. These methods produce accurate results but suffer from the computational effort required which increases rapidly with system size. DFT scales as $\mathcal{O}(N^3)$ and CCSD(T) as $\mathcal{O}(N^7)$ [4], where N represents the system size which can be the number of electrons, atoms or basis functions. These scalings make these methods infeasible for large molecules or high volume molecular screening commonly required in areas such as spintronics and materials design.

Large molecule datasets are generated using these expensive *ab initio* calculations for many fixed nuclear configurations. These produce a mapping between atomic geometry and molecular energy known as the potential energy surface. Machine learning models can be trained to learn the relationships between such atomic geometries and energies, bypassing the need to solve the underlying quantum mechanical equations entirely [4]. Once trained, these models can predict molecular properties at a fraction of the computational cost of the traditional methods, enabling large-scale molecular screening and the exploration of previously inaccessible regions of chemical space. This work investigates the extent to which feedforward neural networks and graph neural networks can learn accurate approximations of the potential energy surface for a range of molecules of varying complexity.

2 Neural Networks

2.1 Feed Forward Neural Networks

Feedforward neural networks are a class of machine learning model commonly used for supervised learning tasks such as regression and classification [5]. Multilayer feedforward neural

networks are capable of approximating non-linear functions providing a flexible framework for mapping geometric descriptors to potential energy. A neural network consists of an input layer, one or more hidden layers and an output layer. A multi-feature input is represented as a column vector $\vec{x} = (x_1, x_2, x_3)^\top$, where each x_i corresponds to a node in the input layer.

This input is forward propagated into the first hidden layer where we apply a linear transformation using a weight matrix $\mathbf{W^{(1)}}$ and a bias $\vec{b}^{(1)}$, followed by an activation function applied element wise to each component of the output vector. These weight matrices are randomly initialized at the start and are trained over many epochs of the model. Activation functions introduce non-linearity which enable the network to learn complex, non-linear relationships.

$$\vec{z}^{(1)} = \mathbf{W^{(1)}}\vec{x} + \vec{b}^{(1)} \longrightarrow \vec{a}^{(1)} = \sigma(\vec{z}^{(1)}), \qquad \mathbf{W^{(1)}} \in \mathbb{R}^{n\times 3} \tag{2}$$

This same operation is applied across each subsequent hidden layer. Each row in the weight matrix corresponds to the incoming connections between one layer and a particular neuron in the next layer.

$$\sigma(\mathbf{W^{(2)}}\vec{a}^{(1)} + \vec{b}^{(2)}) = \vec{a}^{(2)}, \qquad \mathbf{W^{(2)}} \in \mathbb{R}^{n\times n} \tag{3}$$

$$a_0^{(2)} = \sigma(w_{00}^{(2)}a_0^{(1)} + w_{01}^{(2)}a_1^{(1)} + \cdots + w_{0n}^{(2)}a_n^{(1)}) \tag{4}$$

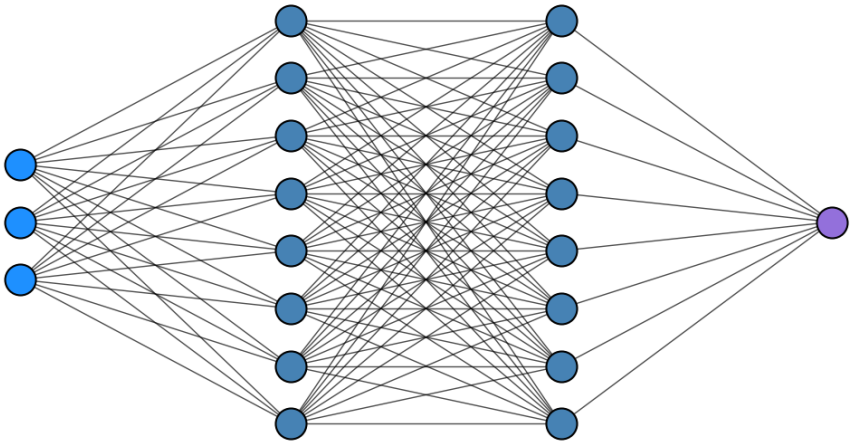

Figure 1: Example of neural network framework

This same process can be repeated for a number of hidden layers of differing sizes and different activation functions. The number of layers, nodes per layer, and choice of activation function is what we call the architecture of the network. These are hyperparameters that must be tuned to optimise the model's predictive performance for a given task. The output layer follows the same structure as the rest of the network, except the weight matrix $\mathbf{W^{(n)}}$ projects to the desired output dimension. If our network were to output a scalar:

$$\vec{z}^{(n)} = \mathbf{W^{(n)}}\vec{a}^{(n)} + \vec{b}^{(n)} \longrightarrow \hat{y} = \sigma(\vec{z}^{(n)}), \qquad \mathbf{W^{(n)}} \in \mathbb{R}^{1\times n} \tag{5}$$

The full forward pass can be written as a single nested function:

$$\hat{y} = \sigma\left(\mathbf{W^{(n)}}\sigma\left(\mathbf{W^{(n-1)}}\sigma\left(\ldots\sigma\left(\mathbf{W^{(1)}}\vec{x} + \vec{b}^{(1)}\right)\right) + \vec{b}^{(n-1)}\right) + \vec{b}^{(n)}\right) \tag{6}$$

This forward pass is used identically in training and testing. During training, the network's predictions are compared against the target energies through a loss function, with back propagation used to update the model's parameters accordingly.

2.2 Back Propagation

For a regression task with target value $y \in \mathbb{R}$, we define a loss function. A common choice is mean squared error (MSE):

$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2 \tag{7}$$

Back propagation computes the gradient of the loss with respect to each model parameter, enabling gradient-based optimization to minimize the loss. To compute how the loss depends on $\mathbf{W^{(1)}}$, the chain rule is applied across the computational graph, since the forward pass dependencies are:

$$\mathbf{W^{(1)}} \to \vec{z}^{(1)} \to \vec{a}^{(1)} \to \vec{z}^{(2)} \to \vec{a}^{(2)} \to \cdots \to z^{(n)} \to a^{(n)} = \hat{y} \tag{8}$$

By chain rule, the exact scalar gradient of the loss with respect to the weight $W_{ik}^{(1)}$ in a network of n layers is given by:

$$\left(\frac{\partial \mathcal{L}}{\partial W_{ik}^{(1)}}\right) = \left(\frac{\partial \mathcal{L}}{\partial a^{(n)}}\right)\left(\frac{\partial a^{(n)}}{\partial z^{(n)}}\right)\left(\frac{\partial z^{(n)}}{\partial a^{(n-1)}}\right)\cdots\left(\frac{\partial z^{(2)}}{\partial a^{(1)}}\right)\left(\frac{\partial a^{(1)}}{\partial z^{(1)}}\right)\left(\frac{\partial z^{(1)}}{\partial W_{ik}^{(1)}}\right) \tag{9}$$

This chain of partial derivatives propagates the error signal backward through the network.

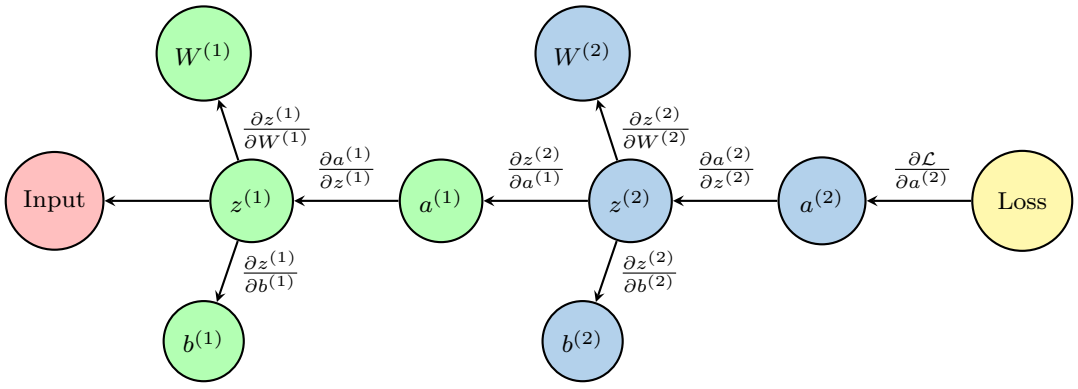

Figure 2: Back propagation chain rule through a two-layer network illustrating Eq. (9) for $n = 2$. $\mathbf{W^{(l)}}$ and $\vec{b}^{(l)}$ denote the full weight matrix and bias vector of layer l which encompass all connections between layers $l-1$ and l. The partial derivatives shown on each arrow represent the local gradient contributions accumulated during back propagation.

Each weight is then updated in the direction that locally decreases the loss through a process called gradient descent, shown here for $W_{ik}^{(1)}$,

$$W_{ik}^{(1)} \leftarrow W_{ik}^{(1)} - \eta \left(\frac{\partial \mathcal{L}}{\partial W_{ik}^{(1)}} \right), \qquad \eta \equiv \text{Learning rate} \tag{10}$$

Forward propagation and back propagation are repeated over many epochs using batches of training data until the loss converges to a minimum.

3 Graph Neural Networks

Graph neural networks (GNN) are an advanced form of neural networks that restructure input data into topological information [6, 7]. They consist of nodes, which represent some real world object, and edges which represent the interactions or relations between these objects.

Molecules are inherently graphs, with atoms serving as nodes and edges encoding the spatial relationships between them such as interatomic distances and angles. This structure motivates the use of GNNs over standard feedforward neural networks for molecular property prediction.

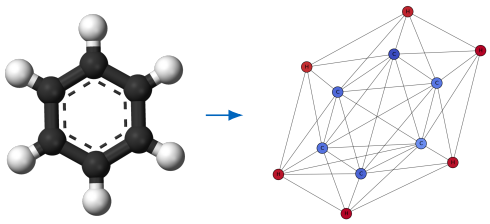

Figure 3: Benzene molecule to GNN representation

A standard feedforward neural network operates on a fixed size, fully connected input vector. Every input feature is connected to every node in the first hidden layer with information flowing in one direction from input to output. This architecture is well suited for tabular data but struggles for graph-structured data like molecules [7]. One could flatten or concatenate the atomic coordinates into a vector and feed them directly into a feedforward network, however this destroys the relational structure of the molecule entirely. This approach would require a fixed input size which isn't realistic when working across a dataset of different molecules where the number of atoms changes between samples. Furthermore, while atomic coordinates do encode geometric information such as interatomic distances and neighbourhood proximity, they carry no information about which neighbouring atoms share electron density through a chemical bond. Not all atoms in spatial proximity of each other are chemically bonded, and a flat coordinate vector, even after geometric encoding, provides no means of distinguishing bonded from non-bonded interactions.

A GNN resolves this by replacing the single, fully connected network with a collection of smaller neural networks that operate locally and simultaneously across every node in the graph. Each atom maintains its own feature vector and runs the same shared neural network to aggregate chemical information exclusively from its nearest neighbors by propagating this information through edges. Through multiple iterations of these neural networks, each atom can learn the chemically relevant information contained within its nearest neighbor range necessary for whatever molecular property we are trying to predict. This process repeats

across multiple layers, progressively allowing atoms across the graph to have an influence on each other even if they aren't nearest neighbors.

The key architectural distinction from a classical neural network and a GNN is the local and parallel nature of computation. A further consequence of this design is permutation invariance which is key for molecular modeling. Once the message passing layers have been completed, each of the atoms transformed feature vectors are aggregated for a final molecular prediction in an operation known as pooling. This is usually done through a summation, mean summation or max pooling, all of which are independent of the order in which the atoms are listed.

3.1 Embedding

Atomic number is the fundamental one-body invariant feature. Raw atomic numbers carry no geometric or relational meaning. To address this, we map each atomic number to a continuous learnable vector representation via an embedding matrix [8, 7], $\mathbf{E} \in \mathbb{R}^{N_{types} \times d}$, where N_{types} is the number of distinct atoms we may need and d is the embedding dimension. Each row $\mathbf{E}(Z)$ corresponds to an atom type and contains a learnable feature vector. The matrix is randomly initialized at the start of training, for example:

$$\mathbf{E} = \begin{bmatrix} \mathbf{E}(H) \\ \vdots \\ \mathbf{E}(C) \\ \mathbf{E}(N) \\ \mathbf{E}(O) \\ \vdots \end{bmatrix} = \begin{bmatrix} \mathbf{E}(1) \\ \vdots \\ \mathbf{E}(6) \\ \mathbf{E}(7) \\ \mathbf{E}(8) \\ \vdots \end{bmatrix} = \begin{bmatrix} 0.12 & -0.67 & \cdots & 0.93 \\ \vdots & \vdots & \ddots & \vdots \\ 0.04 & 0.32 & \cdots & -0.51 \\ -0.39 & 0.18 & \cdots & 0.91 \\ 0.72 & -0.44 & \cdots & -0.18 \\ \vdots & \vdots & \ddots & \vdots \end{bmatrix} \tag{11}$$

When a molecule is processed, the model performs an embedding lookup in $\mathbf{E}$ for each atom, assembling a node feature matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, where N is the number of atoms in the molecule. For example take Formaldehyde (CH_2O):

$$\mathbf{X} = \begin{bmatrix} \mathbf{E}(6) \\ \mathbf{E}(1) \\ \mathbf{E}(1) \\ \mathbf{E}(8) \end{bmatrix} = \begin{bmatrix} 0.04 & 0.32 & \cdots & -0.51 \\ 0.12 & -0.67 & \cdots & 0.93 \\ 0.12 & -0.67 & \cdots & 0.93 \\ 0.72 & -0.44 & \cdots & -0.18 \end{bmatrix} \tag{12}$$

These initial node features are updated through message passing layers, where messages from neighboring nodes are propagated along edges and aggregated to the node embedding. Rather than manually specifying physical properties such as electronegativity, atomic radius, or ionization energy, the model is free to determine what each dimension of the embedding encodes during training. In this sense, the learned embedding dimensions may implicitly correspond to chemically meaningful quantities, though they need not align with any single interpretable quantity. Each entry in the embedding simply represents what the model finds most useful for predicting the target quantity.

During backpropagation, gradients flow back through the network and update the embedding rows corresponding to the atom types present in the current training dataset. For Formalde-

hyde, $\mathbf{E}(1)$, $\mathbf{E}(6)$, and $\mathbf{E}(8)$ are updated based on the prediction error. Rows corresponding to atom types absent from the dataset, such as $\mathbf{E}(7)$ for Nitrogen, remain unchanged. Thus, over many training iterations across a diverse training set of molecules, each embedding vector $\mathbf{E}(Z)$ converges toward a representation that encodes chemically meaningful properties of atom type Z.

3.2 Two-Body Features

Message passing becomes more prominent when incorporating two-body features, which uses pairwise atomic interactions and the embeddings. During encoding, we construct an embedding matrix $\mathbf{E}$ as before. Additionally, we compute the Euclidean distances between every pair of atoms in the molecule. For atoms i and j whose positions are $\vec{r}_i$ and $\vec{r}_j$:

$$d_{ij} = ||\vec{r}_i - \vec{r}_j|| \tag{13}$$

The edge list connecting atom i to atom j is built using a cut-off criterion of $d = 3.3$Å. Only atom pairs with $d_{ij} \leq 3.3$Å are connected by edges. The cut-off captures chemically relevant interactions since atoms beyond this distance typically have negligible influence on local properties. This also connects each atom to its nearest neighbors.

Rather than using raw distances directly, we convert each distance into smooth, learnable representations using radial basis functions (RBFs) [8]. These distributions allow the model to better distinguish between different interatomic distances,

$$\phi_k(d_{ij}) = \exp(-\gamma(d_{ij} - \mu_k)^2), \qquad k = 1, ..., R \tag{14}$$

where μ_k are centers uniformly spaced in the interval $[0, 3.3\text{Å}]$, with γ controlling the width of each Gaussian function. Each distance d_{ij} is thus expanded into a vector of R RBF values.

$$d_{ij} \longrightarrow [\phi_1(d_{ij}), \phi_2(d_{ij}), \ldots, \phi_R(d_{ij})] = \text{rbf}_{ij} \in \mathbb{R}^R \tag{15}$$

Each edge in the GNN now carries a feature vector rbf_{ij} of length R. Each of these vectors form an RBF matrix $\mathbf{G} \in \mathbb{R}^{E \times R}$, where E is the number of edges in the molecule. This representation is rotationally and translationally invariant as it only depends on the magnitude of d_{ij}.

Once distances have been encoded as RBF vectors, we can perform message passing. In each message passing layer, information flows along edges to update node representations. Within message passing, there are multiple atom-wise and bond-wise linear transformations to be applied as well as activation functions. We use the sigmoid linear unit (SiLU) [9] for the activation as it is a smooth function that is differentiable everywhere and most closely represents a mapping from the atomic positions to the total energy of the molecule. It is given by,

$$\sigma(x) = \frac{x}{1 + e^{-x}} \tag{16}$$

Atom-wise transformations intake the embedding vector whereas the bond-wise transformations intake the edge information containing the RBF vector. A typical pipeline would look like,

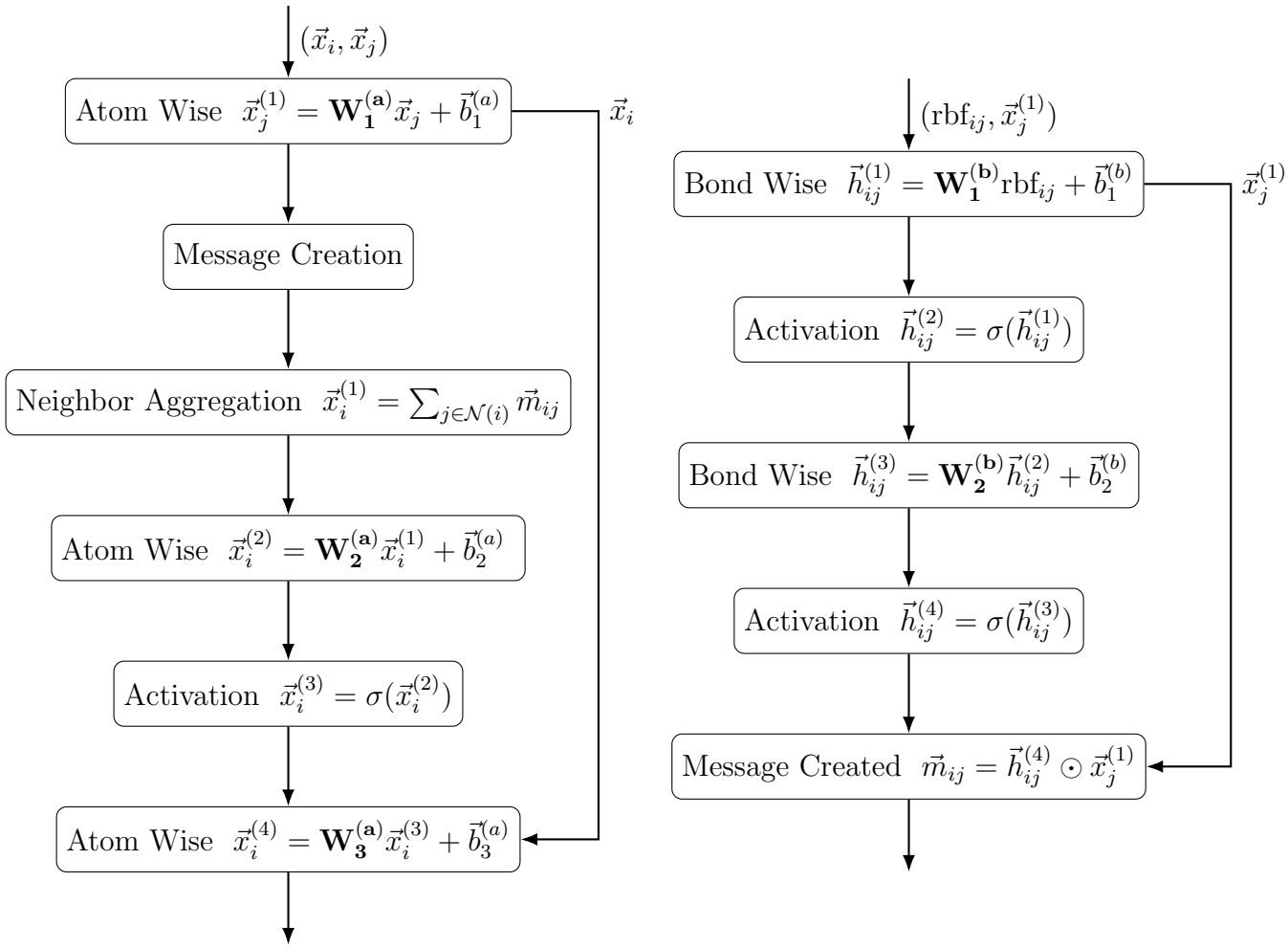

Figure 4: Message passing pipeline

At the end of the message passing the node i is updated;

$$\vec{x}_i^{(t+1)} = \vec{x}_i^{(t)} + \vec{x}_i^{(4)} \tag{17}$$

The updated node features $(\vec{x}_i^{(t+1)}, \vec{x}_j^{(t+1)})$ are then propagated to the next message passing layer, where the cycle of neighbor aggregation and feature updating repeats. The process repeats iteratively for a fixed number of layers for every node in the model, allowing information to propagate throughout the entire graph. After completing all message passing layers, the final node embeddings $\vec{x}_i^{(T)}$ encode local atomic environments and a global molecular structure [7].

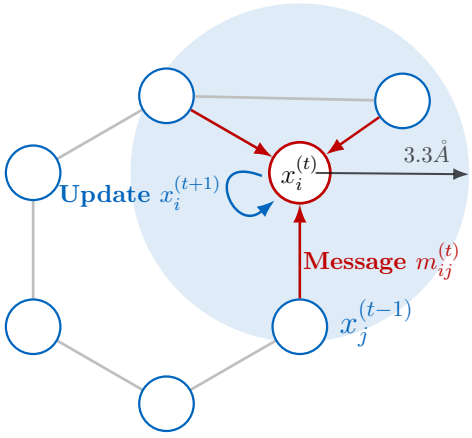

Figure 5: Update scheme

To obtain a molecular prediction from these node representations we employ a readout layer. Each node $\vec{x}_i^{(T)} \in \mathbb{R}^R$ is passed through a sequence of linear transformations and activations parameterized by weight matrices $\mathbf{W}_1^{(r)} \in \mathbb{R}^{K\times R}$ and $\mathbf{W}_2^{(r)} \in \mathbb{R}^{1\times K}$, which project the node embeddings down to a scalar contribution. The intermediate dimension K acts as a bottleneck, compressing the atomic representation before projecting to a scalar and is treated as a hyperparameter in training. A global pooling operation sums these scalar contributions across all atoms in the molecule which gives a final predicted molecule energy E_{pred}. This summation is permutation invariant and is required by molecular property prediction as the total energy is independent of atom ordering. Summation pooling is chosen over mean or max pooling as the predicted energy scales naturally with the number of atoms in the molecule which is consistent with how molecular energies physically behave.

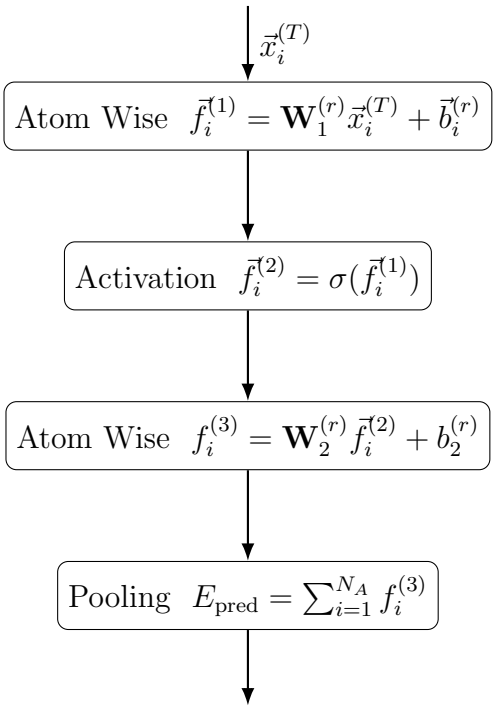

Figure 6: Readout pipeline

3.3 Three-Body Features

To enhance the model's sensitivity to directional arrangements of neighboring atoms, we encode angle features into our edge representations. Each angle is defined by a triplet of atoms $(\vec{r}_i, \vec{r}_j, \vec{r}_k)$, where atoms j and k are both within the cut-off range of the central atom i.

The angle θ_{jik} between bonds $\vec{r_{ij}}$ and $\vec{r}_{ik}$ is computed via the dot product, where $d_{ij} = ||\vec{r}_i - \vec{r}_j||$ and $d_{ik} = ||\vec{r}_i - \vec{r}_k||$ are the pairwise distances. The triplet $(d_{ij}, d_{ik}, \theta_{jik})$ uniquely specifies the local geometry around atom i and is invariant under rotation and translation.

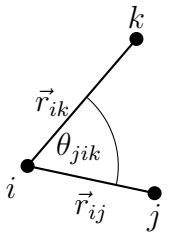

Figure 7: Angle θ_{jik} formed by atoms j–i–k.

Analogous to the RBF treatment of distances, we expand each angle into a learnable representation to introduce nonlinearity. We use a Legendre polynomial basis [10] $\{P_l(\cos(\theta_{jik})\}_{l=1}^{S}$, computed via the recurrence relation,

$$P_{l+1}(\cos\theta) = \frac{(2l+1)P_l(\cos\theta)\cos\theta - lP_{l-1}(\cos\theta)}{l+1} \tag{18}$$

with initial conditions $P_0(\cos\theta) = 1$ and $P_1(\cos\theta) = \cos\theta$. We omit the $l = 0$ term as it is a constant. This gives a spherical basis function (SBF) vector for each angle which we match

with the dimensionality of the RBF expansion.

$$\theta_{jik} \longrightarrow [P_1(\cos\theta_{jik}), P_2(\cos\theta_{jik}), \ldots, P_S(\cos\theta_{jik})] = \text{sbf}_{jik} \in \mathbb{R}^S \tag{19}$$

A key challenge with incorporating angles is that they are inherently three-body interactions and our message passing network was designed to operate on two-body edges. To reconcile this, we construct a SBF matrix $\mathbf{M} \in \mathbb{R}^{E\times S}$, where E is the number of edges per molecule (bond pairs within the cut-off). Initially, all entries in $\mathbf{M}$ are initialized to zero. For each angle θ_{jik} formed by edges (i, j) and (i, k), we identify the row indices corresponding to edges (i, j) and (i, k) in $\mathbf{M}$ and then add the SBF vector sbf_{jik} to both rows. This aggregation ensures that each edge accumulates angular information from all angles in which it participates. Edges involved in multiple angles receive the sum of their corresponding SBF vectors. Finally the RBF matrix $\mathbf{G} \in \mathbb{R}^{E\times R}$ and the SBF matrix $\mathbf{M} \in \mathbb{R}^{E\times S}$ are concatenated column-wise to form the complete edge feature matrix:

$$\mathbf{F} = [\mathbf{G}|\mathbf{M}] \in \mathbb{R}^{E\times(R+S)} \tag{20}$$

This feature matrix is then propagated through the message passing layers. The weight matrices in the bond-wise transformation layers are adjusted accordingly to accommodate for the increased feature dimension.

4 Results

4.1 Training

The feedforward neural network, two-body, and three-body GNN were trained under a consistent set of hyperparameters to ensure a fair comparison. We use 2000 configurations per molecule drawn from the revised MD17 (rMD17) dataset [11], partitioned into a training set of 1600 and a testing set of 400. Prior to training, all target energies were normalised using the mean and standard deviation computed exclusively from the training set. The same statistics were used to normalise the test set, ensuring the model had no exposure to the statistical properties of the test data preventing data leakage.

Each model was trained for 200 epochs. A single epoch consists of one complete pass of all training samples. Molecules were processed in a batch size of 32 for computational efficiency and to introduce stochasticity into gradient updates [5]. Rather than computing the exact gradient over the full training set, we compute the average gradient across 32 samples. This introduces a degree of noise that acts like an implicit regulariser, allowing the model to generalise better to unseen molecules. This way, each weight gets updated 50 times per epoch as opposed to just once through gradient descent. All models were optimised using the Adam optimiser [12], an adaptive gradient descent tool that adjusts the learning rate for each parameter individually based on gradient history, resulting in faster convergence.

Hyperparameter	Value	Applicable Models
Learning Rate	$1 \times 10^{-7} < \eta < 1 \times 10^{-3}$	All
Weight Decay	1×10^{-4}	All
Training Set	1600	All
Testing Set	400	All
Batch Size	32	All
Epochs	200	All
Activation	SiLU	All
Message Passing Layers	3	GNN
Dropout	0.1	GNN

Table 1: Training hyperparameters

A cosine annealing learning rate scheduler was employed [13], adjusting the global learning rate between a minimum of 1×10^{-7} and a maximum of 1×10^{-3}.

$$\eta_{t+1} = \eta_{min} + (\eta_t - \eta_{min}) \cdot \frac{1 + \cos\left(\frac{(t+1)\pi}{T_{max}}\right)}{1 + \cos\left(\frac{t\pi}{T_{max}}\right)} \tag{21}$$

This allows the model to make weight adjustments with large steps initially to escape poor local minima, then gradually decreases the step size to allow for fine tuning. A weight decay of 1×10^{-4} was used as another regularisation penalty. This discourages large weights by shifting them towards zero regardless of the gradient. This helps prevent overfitting as large weights are a symptom of a model memorising specific input features of the training data. SiLU was used as the activation function throughout, as introduced in section 3.2.

For the GNN architectures, three message passing layers were used allowing atomic representations to aggregate information from neighbors up to three edges away. A dropout of 0.1 was used [14]. During each forward pass in training, dropout randomly zeroes a fraction of the nodes outputs. This prevents the model from relying too heavily on specific nodes in the graph to make predictions, reducing overfitting.

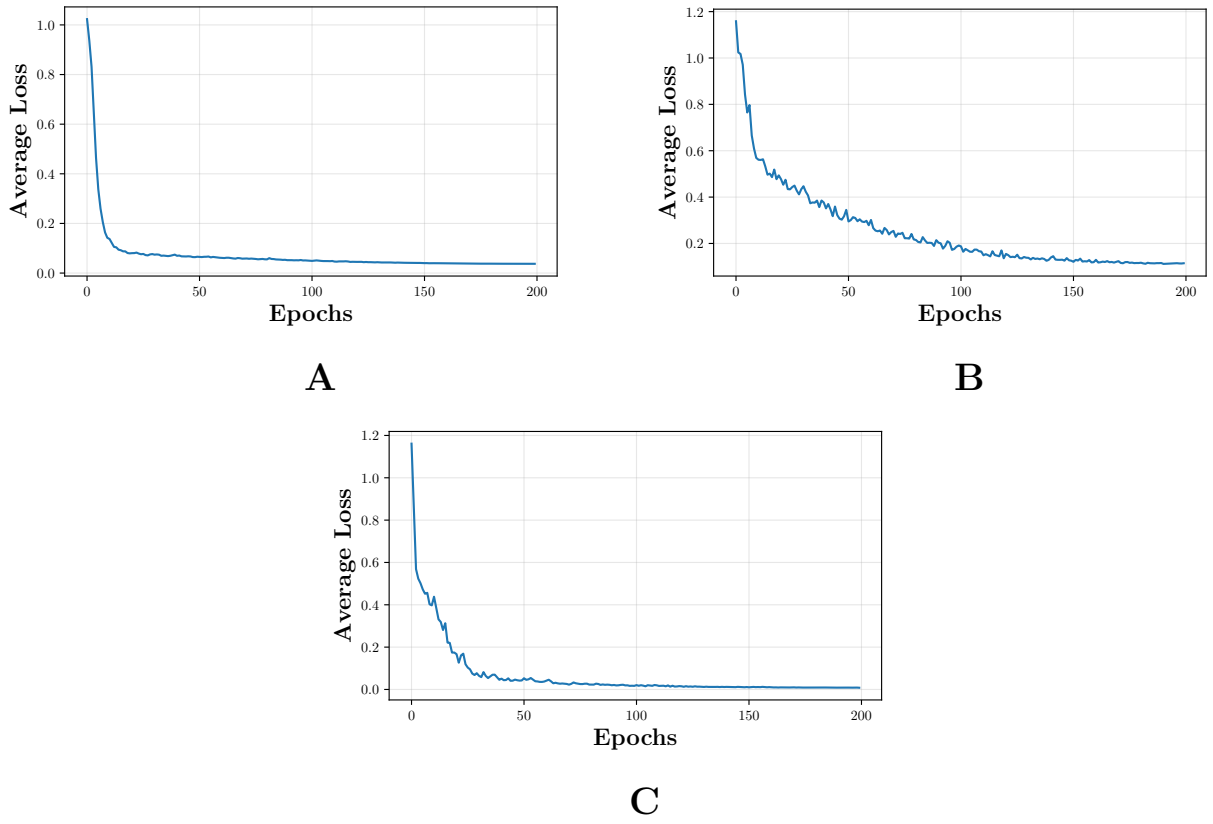

Figure 8: Training loss vs. epoch for **(A)** Feedforward Neural Network, **(B)** Two-body GNN, and **(C)** Three-body GNN for Benzene

Training loss curves were generated for each model evaluated on the benzene molecule. All three models show steady convergence with the loss dropping significantly within the first 25 epochs and gradually plateauing as the scheduler reduces the learning rate towards a minimum. The feedforward network exhibits a smoother convergence curve than both GNN models, which can be attributed to the nature of its input features. The model is tailored specifically to benzene, with bond lengths and angles explicitly defined as a fixed feature vector, leaving no ambiguity in the model's calculations. The fully connected nature of the feedforward network also ensures each parameter receives a stable gradient update influenced by all input features simultaneously. This results in consistent, well-conditioned gradient updates across batches. By contrast, the GNN models must learn molecular representations from scratch through message passing over RBF and SBF encoded interatomic features. Gradients must propagate back through atomic embeddings and edge networks in sparse local neighborhoods over multiple message passing layers introducing additional complexity to the loss curves.

4.2 Testing

Following training, each model was evaluated on their respective test sets of 400 molecule configurations. Predicted energies were unnormalised back to their original units and plotted against true target energies. Perfect predictions lie along the diagonal with deviations indicating prediction error.

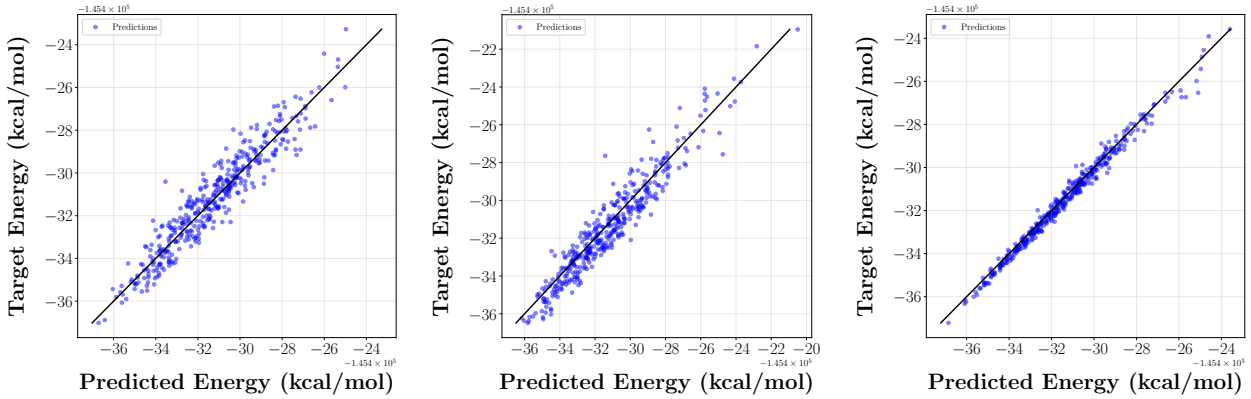

Figure 9: Parity plots of predicted vs. target energy for benzene across all three models: **(A)** Feedforward Neural Network, **(B)** Two-body GNN, and **(C)** Three-body GNN.

For benzene, all three models generalise well to unseen data, producing tight clustering lines around the diagonal. There is minimal improvement from the feedforward neural network to the two-body GNN, which is consistent with the feedforward model being explicitly tailored to benzene's geometry and the two-body GNN lacking full spatial encoding. The three-body GNN performs the strongest with predictions closely aligned with the diagonal across almost all samples. This strong performance of all three models reflects benzene's inherent simplicity. Its symmetric ring structure and uniform bonding environment produce a relatively smooth energy landscape that each model can capture effectively.

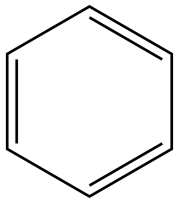

Figure 10: Molecular structure of benzene

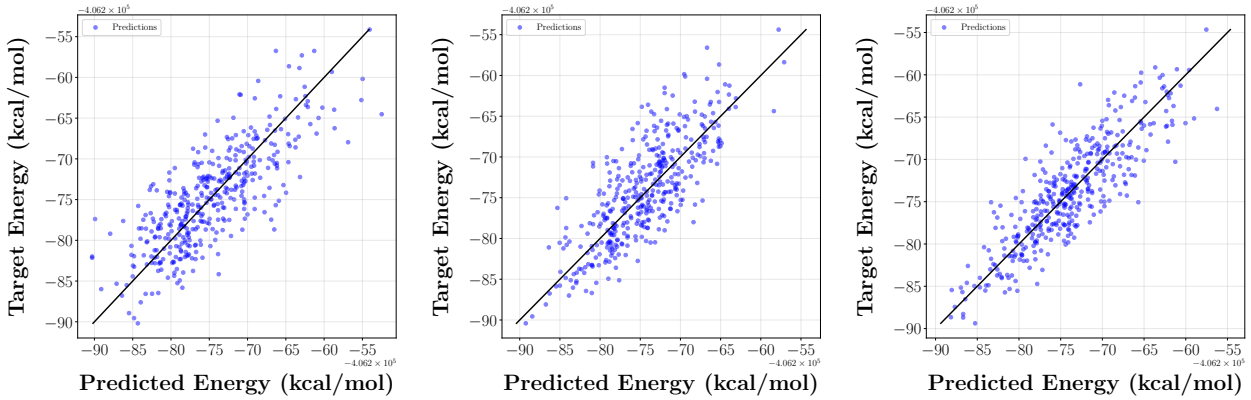

Figure 11: Parity plots of predicted vs. target energy for Aspirin across all three models: **(A)** Feedforward Neural Network, **(B)** Two-body GNN, and **(C)** Three-body GNN.

For aspirin, all three models show considerably more scatter around the diagonal, reflecting the complexity of aspirin's energy surface. The three-body GNN processes this complexity the best, with the tightest clustering of the three. Unlike benzene, the feedforward neural network performs the poorest. Aspirin's varied bond types, such as carbonyl double bonds, ester linkages, and aromatic bonds, cannot be explicitly encoded into the feedforward neural network's feature vector beyond their geometric footprint [15]. This leaves the model unaware of the underlying electronic character of each bond. The GNNs can implicitly learn to distinguish these atomic relationships much better through learning continuous, high-dimensional representations of each atom's local chemical environment through iterative message passing [7]. This gives the GNNs freedom to encode bond order, electron density and influences from neighbouring atoms in a way fixed geometric descriptors cannot, proving themselves much better equipped for the task.

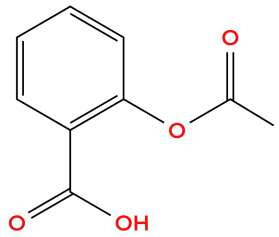

Figure 12: Molecular structure of aspirin

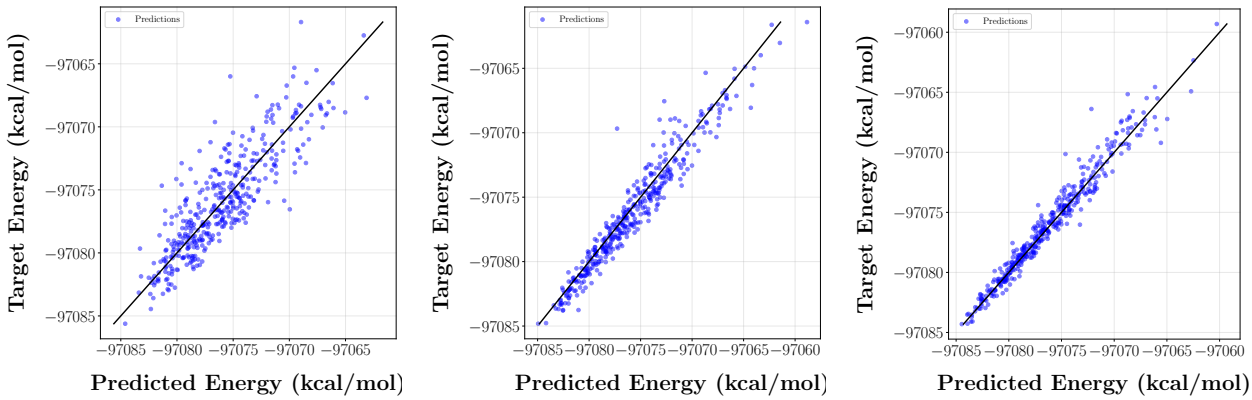

Figure 13: Parity plots of predicted vs. target energy for Ethanol across all three models: **(A)** Feedforward Neural Network, **(B)** Two-body GNN, and **(C)** Three-body GNN.

Ethanol represents an intermediate case between the simplicity of benzene and the complexity of aspirin. As expected, the GNNs outperform the feedforward network with the three-body GNN producing the tightest scattering around the diagonal. Ethanol's bonding environment is not uniform, with the presence of a hydroxyl group introducing a polar C-O bond. The electronic character of these atomic relationships extend beyond what a geometric descriptor can capture. This reinforces the central limitation of feedforward neural networks. They can remain competitive with GNNs when every bond in the molecule can be completely characterised by hand, but fall short when the bonding environment introduces chemical diversity that a fixed feature vector cannot adequately express.

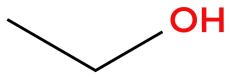

Figure 14: Molecular structure of ethanol

4.3 Model Performance

Table 2 shows the final training and testing losses across benzene, aspirin and ethanol alongside a ratio of test to training loss, with a value around 1 indicating good generalisation.

Molecule	Model	Train Loss	Test Loss	Ratio
Benzene	Feedforward NN	0.0380	0.1171	3.08
	Two-body GNN	0.1036	0.1573	1.52
	Three-body GNN	0.0117	0.0144	1.23
Aspirin	Feedforward NN	0.0110	0.5746	52.2
	Two-body GNN	0.1379	0.3298	2.39
	Three-body GNN	0.0083	0.3045	36.7
Ethanol	Feedforward NN	0.0956	0.2767	2.89
	Two-body GNN	0.0366	0.0633	1.73
	Three-body GNN	0.0063	0.0389	6.17

Table 2: Final training and test losses for each model and molecule

The loss ratios reveal significant overfitting on aspirin for the feedforward network and three-body GNN, with ratios of 52.2 and 36.7 respectively. This indicates the model has memorised specific geometric configurations in the training set rather than learning a generalisable representation of its structure. Despite this high ratio, the three-body GNN achieves a lower test loss than the two-body model on aspirin. This suggests that the two-body GNN did not fit the training data well to begin with, artificially deflating its ratio without necessarily indicating better generalisation.

Table 3 presents the performance metrics for the feedforward model. R^2 is the coefficient of determination, measuring the proportion of variance in the target energies explained by the model, ranging from 0 to 1 with 1 indicating a perfect prediction.

Molecule	R^2	**MAE (kcal/mol)**	**RMSE (kcal/mol)**
Benzene	0.8855	0.6306	0.8365
Aspirin	0.4520	3.6081	4.6436
Ethanol	0.7312	1.6697	2.2734

Table 3: Performance metrics for the feedforward neural network

The feedforward network performs best on benzene, achieving an R^2 of 0.8855 and a mean absolute error (MAE) of 0.6306 kcal/mol which is consistent with the tight clustering observed in the parity plot. Performance decreases for ethanol and aspirin reflecting the inability of fixed geometric descriptors to capture chemical diversity as the molecular complexity increases.

	Two-body GNN			**Three-body GNN**		
Molecule	MAE	RMSE	R^2	MAE	RMSE	R^2
Benzene	0.7537	0.9962	0.8303	0.2059	0.2841	0.9861
Toluene	1.6422	2.1072	0.8287	0.9565	1.2316	0.9406
Malonaldehyde	1.2967	1.7482	0.8292	1.2798	1.6225	0.8399
Salicylic acid	2.1564	2.7369	0.7791	1.6745	2.0839	0.8508
Aspirin	2.9042	3.6707	0.6286	2.5535	3.4218	0.6916
Ethanol	0.8401	1.1170	0.9225	0.6258	0.8335	0.9630
Uracil	1.4393	1.8716	0.8679	0.9950	1.3212	0.9338
Naphthalene	1.8158	2.2715	0.8445	0.8412	1.0949	0.9646
	kcal/mol			kcal/mol		

Table 4: Performance metrics for two-body vs three-body GNN models across all molecules.

The three-body GNN consistently outperforms the two-body GNN on all benchmark molecules confirming the addition of angular encoding through SBFs provides additional predictive power. The most noticeable improvements are observed for benzene and naphthalene with the MAE reducing by 73% and 54% respectively outlining the effectiveness of angular encoding when it comes to symmetric aromatic ring geometries [10]. Both GNNs and the feedforward neural network struggle with aspirin, with the three-body GNN achieving an R^2 of 0.6916, consistent with it being the most structurally complex molecule evaluated.

To further validate the three-body GNNs performance, the average atom energy contributions were extracted from the model for 50 benzene molecules in figure 15. The model is seen to predict negative energy contributions from the carbon atoms and positive contributions from the hydrogen atoms. This is physically consistent with the electronic structure of benzene as the delocalised electrons across the carbon ring create stable bonds between carbon atoms, giving rise to a negative potential well in these regions. The peripheral hydrogen atoms give smaller, positive energy contributions as they are less tightly bound to the molecular core. The symmetry of these contributions across the six equivalent carbon and hydrogen atoms confirms the model has learned a physically consistent internal representation of the benzene molecule, rather than arriving at correct predictions through false correlations.

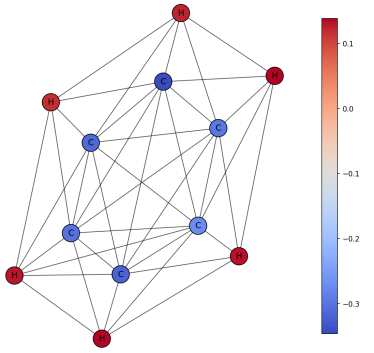

Figure 15: Per-atom energy contributions predicted by the three-body GNN for benzene

4.4 Effect of Training Set Size

To investigate the effect of training set size on model performance, the three-body GNN was retrained using 4000 samples for each molecule with a training set of 3200 and a test set of 800. Parity plots are presented in figures 16 and 17.

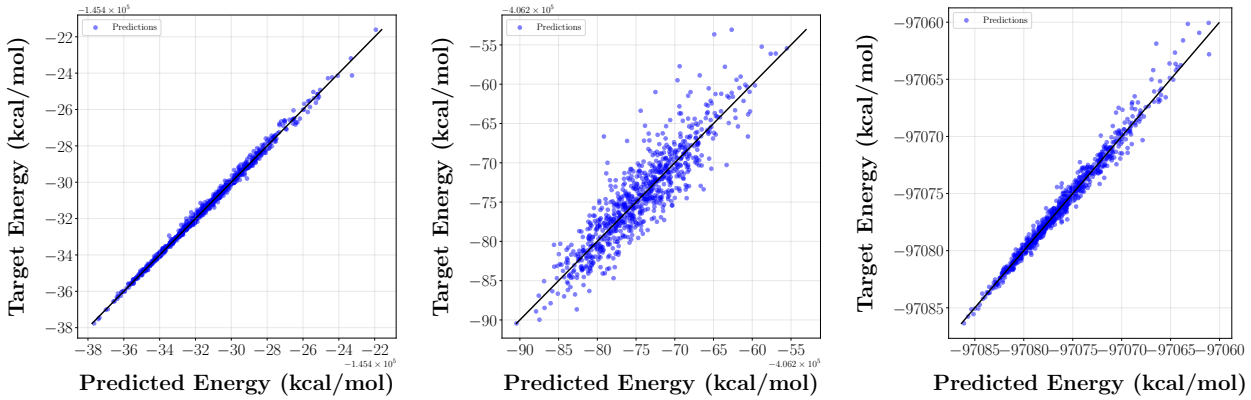

Figure 16: Parity plots of predicted vs. target energy for **(A)** benzene, **(B)** aspirin, and **(C)** ethanol

The improvement over the 2000 sample results is immediately noticeable with tighter clustering around the diagonal across all molecules, with benzene and ethanol showing near perfect alignment. Aspirin shows a notable reduction in scatter relative to figure 11, demonstrating that the three-body GNN's generalisation on complex molecules benefits substantially from increased training set size.

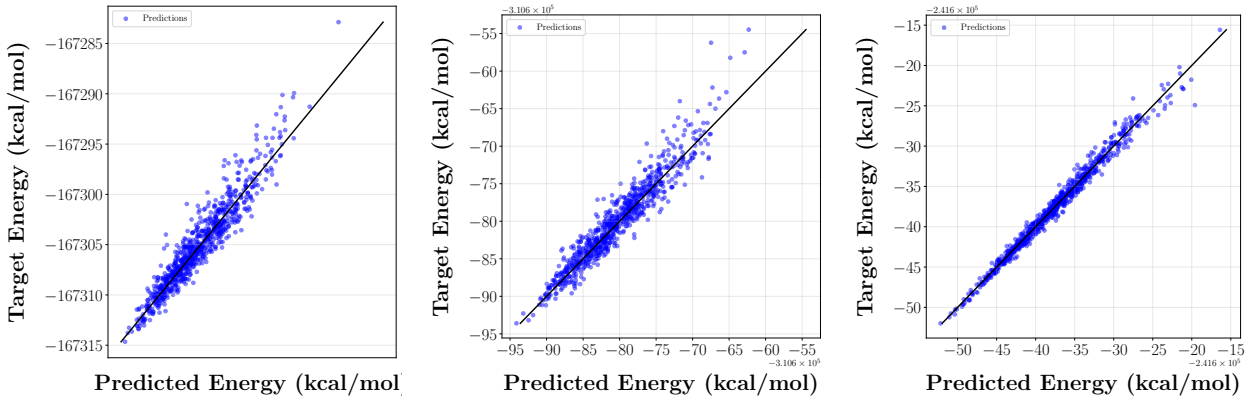

Figure 17: Parity plots of predicted vs. target energy for **(A)** malonaldehyde, **(B)** salicylic acid, and **(C)** naphthalene

Table 5 presents the R^2 and MAE metrics for our three-body GNN trained on 3600 samples from the rMD17 benchmark alongside the MAE produced by SchNet's two-body GNN [8], trained on 1000 samples from the MD17 [16] benchmark.

	Three-Body GNN		**SchNet Two-Body GNN**
Molecule	**R^2**	**MAE**	**MAE**
		kcal/mol	*kcal/mol*
Benzene	0.9958	0.11	1.19
Toluene	0.9687	0.67	2.95
Malonaldehyde	0.8914	1.04	2.03
Salicylic Acid	0.9090	1.26	3.27
Aspirin	0.8039	1.99	4.20
Ethanol	0.9795	0.42	0.93
Uracil	0.9559	0.78	2.26
Naphthalene	0.9848	0.52	3.58

Table 5: Performance comparison between the proposed three-body model and the reference model across the molecular dataset.

The three-body GNN achieves superior MAE across all benchmark molecules, with notable improvements for naphthalene and toluene. These results suggest that the overfitting observed in the 2000 sample model was driven largely by insufficient training data rather than a fundamental limitation of the model architecture. It should be noted that this is not a strictly controlled comparison as the models differ in training set size, benchmark dataset, and architecture. The comparative results highlight the necessity of sufficient training data and angular encoding within a model. As we have demonstrated, our model can produce predictions that are competitive with an established architecture.

5 Conclusions

This report investigated the viability of graph neural networks for molecular potential energy prediction. Three architectures of increasing complexity were developed and evaluated across a benchmark set of molecules from the rMD17 dataset [11] starting with a classical feedforward neural network, a two-body GNN and finishing with a three-body GNN.

The feedforward neural network demonstrated that specifically tailored geometric descriptors can remain competitive with more advanced models for molecules with uniform bonding environments, achieving an R^2 score of 0.8855 for benzene. However, performance dropped considerably for aspirin where an R^2 score of 0.4520 was achieved. This highlighted the fundamental limitation of fixed feature vectors in capturing the electronic character of chemically diverse bonding environments [15].

The two-body GNN demonstrated that learned molecular representations offer greater flexibility for more complex bonding environments, outperforming the feedforward neural network on molecules such as ethanol and aspirin. The three-body GNN consistently outperformed the two-body GNN across all eight benchmark molecules, confirming that the additional angular feature encoding gives the model much more topological awareness and predictive power [10]. The most noticeable improvements were observed for molecules with symmetric aromatic geometries such as benzene and naphthalene, where the MAE decreased by 73% and 54% respectively.

Overfitting was seen as a particular challenge for the models trained on 1600 samples, with the feedforward network and three-body GNN recording a ratio of test to train loss of 52.2 and 36.7 respectively for aspirin. To address this, the training set was doubled to 3200 samples and evaluated on the three-body GNN yielding substantial improvements. This suggests that data insufficiency was the primary cause of overfitting rather than a fundamental architectural limitation. The three-body GNN was then compared to the two-body SchNet [8] GNN from the literature, producing competitive predictions and achieving a lower MAE across all benchmark molecules, though it is noted that this comparison was not strictly controlled.

Per atom contributions extracted from the three-body GNN for benzene confirmed that the model was establishing physically consistent atomic relationships, with carbon atoms contributing negative energies consistent with their stronger C-C bonds and hydrogen atoms contributing small positive energies. This provided confidence that the model has learned physically meaningful atomic representations rather than arriving at energy predictions through artificial correlations.

Future work could explore force prediction alongside energy, requiring the model to learn the gradient of the potential energy surface with respect to the atomic positions. Training on larger, more chemically diverse datasets would further reduce overfitting and improve generalisation. The encoding of four-body interactions such as dihedral angles, which describe the rotational conformation around a bond, may be beneficial for molecules such as aspirin where torsional degrees of freedom play a significant role in predicting the potential energy surface [17].

Bibliography

[1] Clemens C. J. Roothaan. New developments in molecular orbital theory. *Reviews of Modern Physics*, 23(2):69–89, 1951.

[2] Pierre Hohenberg and Walter Kohn. Inhomogeneous electron gas. *Physical Review*, 136(3B):B864–B871, 1964.

[3] Krishnan Raghavachari, Gary W. Trucks, John A. Pople, and Martin Head-Gordon. A fifth-order perturbation comparison of electron correlation theories. *Chemical Physics Letters*, 157(6):479–483, 1989.

[4] Matthias Rupp. Machine learning for quantum mechanics in a nutshell. *International Journal of Quantum Chemistry*, 115(16):1058–1073, 2015.

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[6] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.

[7] Justin Gilmer, Kristof T. Schütt, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1263–1272, 2017.

[8] Kristof T. Schütt, Pieter-Jan Kindermans, Huziel E. Sauceda, Stefan Chmiela, Alexandre Tkatchenko, and Klaus-Robert Müller. Schnet: A continuous-filter convolutional neural network for modeling quantum interactions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[9] Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018.

[10] Johannes Gasteiger, Janek Groß, and Stephan Günnemann. Directional message passing for molecular graphs. In *International Conference on Learning Representations*, 2020.

[11] Anders S. Christensen and O. Anatole von Lilienfeld. On the role of gradients for machine learning of molecular energies and forces. *Machine Learning: Science and Technology*, 1(4):045018, 2020.

[12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[13] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.

[14] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.

[15] Jörg Behler and Michele Parrinello. Generalized neural-network representation of high-dimensional potential-energy surfaces. *Physical Review Letters*, 98(14):146401, 2007.

[16] Stefan Chmiela, Alexandre Tkatchenko, Huziel E. Sauceda, Igor Poltavsky, Kristof T. Schütt, and Klaus-Robert Müller. Machine learning of accurate energy-conserving molecular force fields. *Science Advances*, 3(5):e1603015, 2017.

[17] Johannes Gasteiger, Shankari Giri, Johannes T. Margraf, and Stephan Günnemann. Fast and uncertainty-aware directional message passing for non-equilibrium molecules. In *Machine Learning for Molecules Workshop, NeurIPS*, 2020.

Source Code

The full source code for the models used in this report can be found online at: `https://github.com/staffoeo/Capstone-Models`

Parity Plots

Parity plots for the remaining five benchmark molecules evaluated using the two-body and three-body GNN trained on 1600 samples. These correspond to the performance metrics listed in Table 3.

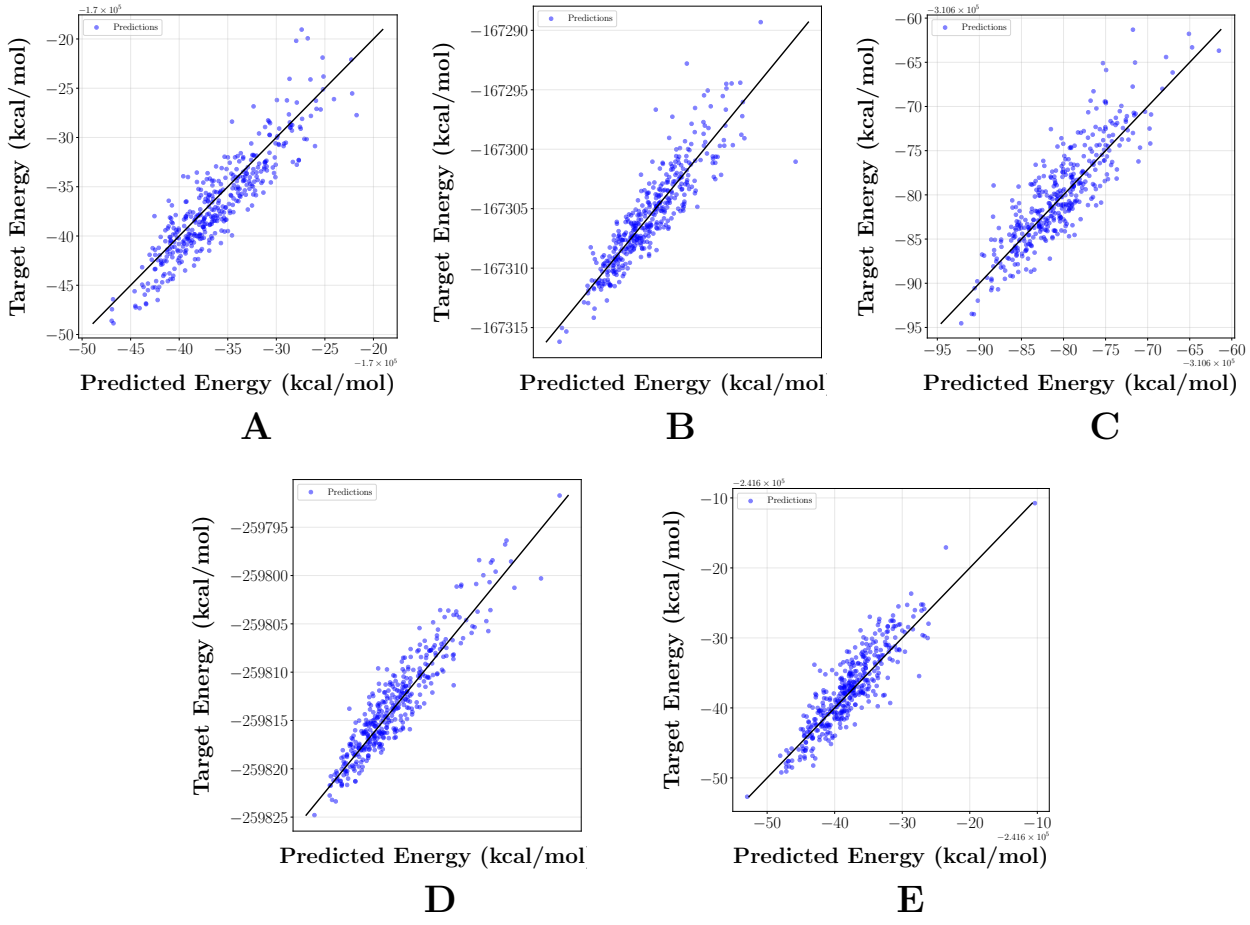

Figure 18: Parity plots of predicted vs. target energy evaluated on the two-body GNN for **(A)** toluene, **(B)** malonaldehyde, **(C)** salicylic acid, **(D)** uracil, and **(E)** naphthalene

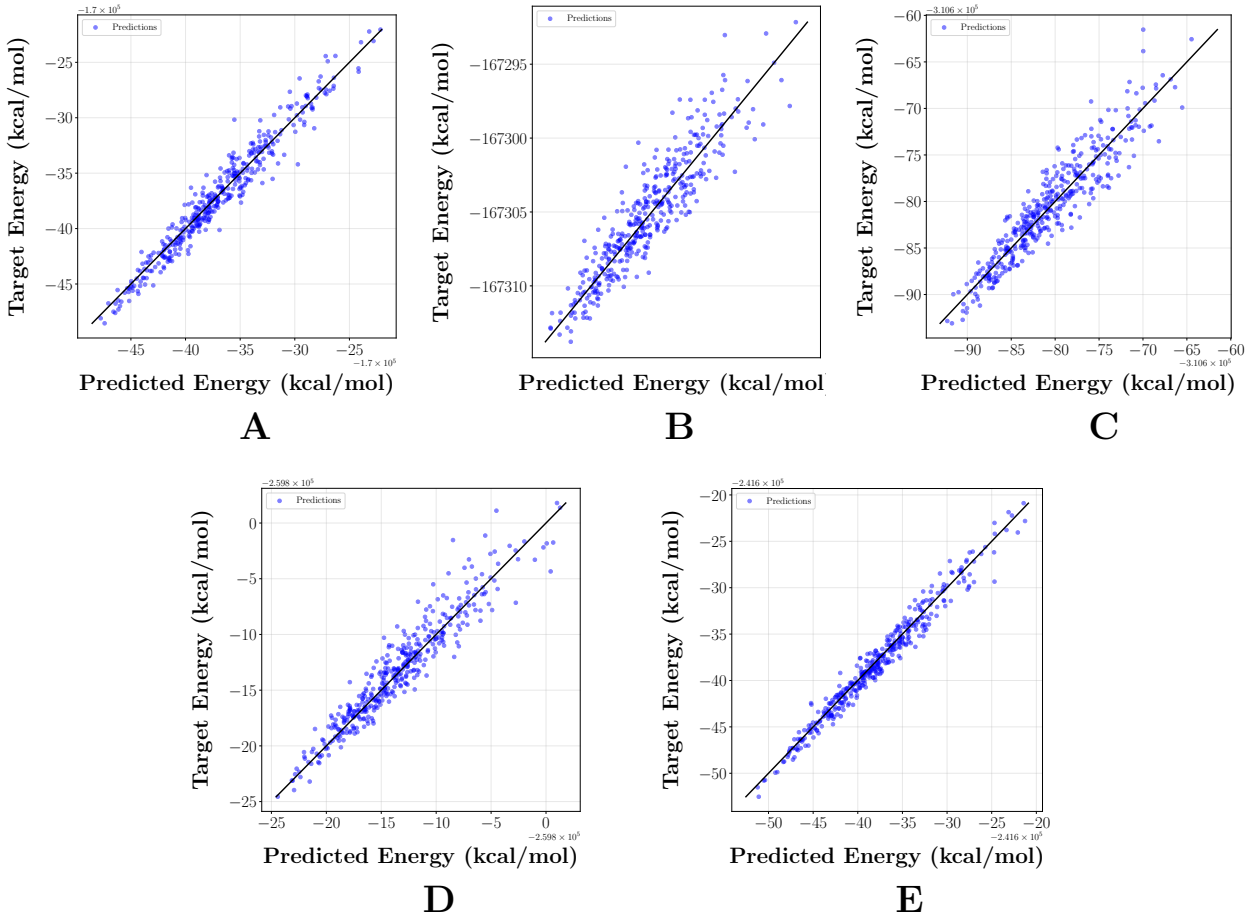

Figure 19: Parity plots of predicted vs. target energy evaluated on the three-body GNN for **(A)** toluene, **(B)** malonaldehyde, **(C)** salicylic acid, **(D)** uracil, and **(E)** naphthalene

The following two parity plots were generated using the three-body GNN trained on 3200 samples for the remaining two benchmark molecules.

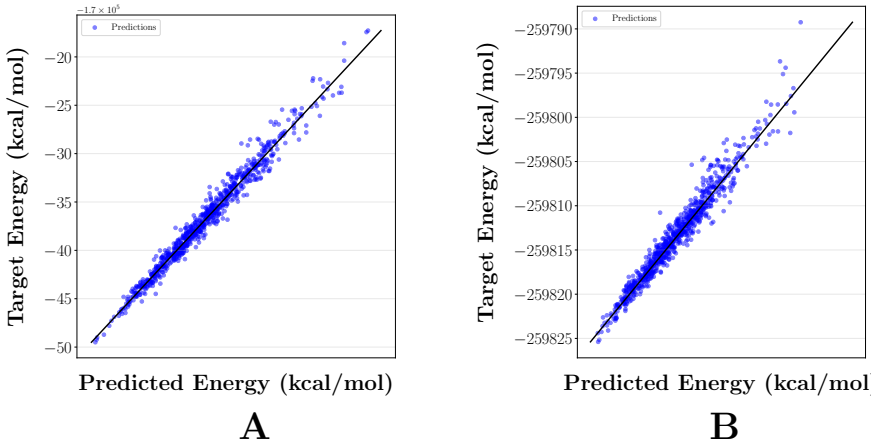

Figure 20: Parity plots of predicted vs. target energy evaluated on three-body GNN for **(A)** toluene **(B)** uracil